

Formación



Línea base Ampliamente disponible *



Este **Array** objeto, al igual que los arreglos en otros lenguajes de programación, permite [almacenar una colección de múltiples elementos bajo un único nombre de variable](#) y cuenta con miembros para [realizar operaciones comunes con arreglos](#).

Descripción

En JavaScript, los arrays no son [tipos de datos primitivos](#), sino **Array** objetos con las siguientes características principales:

- **Los arrays de JavaScript son redimensionables y pueden contener una combinación de diferentes tipos de datos**. (Cuando estas características no sean deseables, utilice [arrays tipados](#)).
- **Los arrays de JavaScript no son arrays asociativos**, por lo que no se puede acceder a los elementos del array utilizando cadenas arbitrarias como índices, sino que se debe acceder a ellos utilizando números enteros no negativos (o su representación en cadena correspondiente) como índices.
- **Los arrays de JavaScript tienen índice cero** [↗](#): el primer elemento de un array está en el índice `0`, el segundo en el índice `1`, y así sucesivamente, y el último elemento está en el valor de la propiedad `1` del array menos `.length`.
- **Las operaciones de copia de matrices en JavaScript crean copias superficiales**. (Todas las operaciones de copia estándar integradas con *cualquier* objeto JavaScript crean copias superficiales, en lugar de [copias profundas](#)).

Índices de matrices

Array Los objetos no pueden usar cadenas arbitrarias como índices de elementos (como en un [array asociativo](#) [↗](#)), sino que deben usar números enteros no negativos (o su representación en cadena). Al establecer o acceder a elementos mediante valores no enteros, no se establece ni se recupera un elemento de la lista del array en sí, sino que se establece o se accede a una variable asociada con la [colección de propiedades del objeto del array](#). [Las propiedades del objeto del array y la lista de elementos del array son independientes, y las operaciones de recorrido y modificación del array no se pueden aplicar a estas propiedades con nombre](#).

Los elementos de un array son propiedades de un objeto de la misma manera que `toString` lo es una propiedad (para ser más precisos, sin embargo, `toString()` es un método). No obstante, intentar acceder a un elemento de un array de la siguiente manera genera un error de sintaxis porque el nombre de la propiedad no es válido:

JS

```
arr.0; // a syntax error
```



La sintaxis de JavaScript requiere que las propiedades que comienzan con un dígito se accedan usando [la notación de corchetes](#) en lugar de [la notación de punto](#) . También es posible entrecomillar los índices de la matriz (por ejemplo, `years[ '2' ]` en lugar de `years[2]`), aunque normalmente no es necesario.

El `2` valor de `in` `years[2]` se convierte en una cadena mediante una `toString` conversión implícita del motor JavaScript. Como resultado, `'2' in` y `'02' in` se referirían a dos ranuras diferentes en el `years` objeto, y el siguiente ejemplo podría ser `true` :

JS

```
console.log(years["2"] !== years["02"]);
```

Only `years[ '2' ]` es un índice real de la matriz. `years[ '02' ]` es una propiedad de cadena arbitraria que no se visitará en la iteración de la matriz.

Relación entre longitud y propiedades numéricas

Las propiedades de propiedad y las propiedades numéricas de un array de JavaScript `length` están relacionadas.

Varios de los métodos de matriz integrados (por ejemplo, `join()` , `slice()` , `indexOf()` , etc.) tienen en cuenta el valor de una `length` propiedad de la matriz cuando se llaman.

Otros métodos (por ejemplo, `push()` , `splice()` , etc.) también dan como resultado actualizaciones de la `length` propiedad de una matriz.

JS

```
const fruits = [];  
fruits.push("banana", "apple", "peach");  
console.log(fruits.length); // 3
```

Al establecer una propiedad en un array de JavaScript cuando la propiedad es un índice válido del array y ese índice está fuera de los límites actuales del array, el motor actualizará la `length` propiedad del array en consecuencia:

JS

```
fruits[5] = "mango";  
console.log(fruits[5]); // 'mango'  
console.log(Object.keys(fruits)); // ['0', '1', '2', '5']  
console.log(fruits.length); // 6
```

Aumentar el tamaño `length` extiende la matriz agregando espacios vacíos sin crear ningún elemento nuevo, ni siquiera `undefined` .

JS

```
fruits.length = 10;
console.log(fruits); // ['banana', 'apple', 'peach', empty x 2, 'mango', empty x 4]
console.log(Object.keys(fruits)); // ['0', '1', '2', '5']
console.log(fruits.length); // 10
console.log(fruits[8]); // undefined
```

length Sin embargo, disminuir el valor de la propiedad sí elimina elementos.

JS

```
fruits.length = 2;
console.log(Object.keys(fruits)); // ['0', '1']
console.log(fruits.length); // 2
```

Esto se explica con más detalle en la **length** página.

Métodos de matriz y espacios vacíos

Los métodos de matriz tienen comportamientos diferentes cuando encuentran ranuras vacías en **matrices dispersas**. En general, los métodos más antiguos (por ejemplo, **forEach**) tratan las ranuras vacías de manera diferente a los índices que contienen **undefined**.

Los métodos que tienen un tratamiento especial para las ranuras vacías incluyen los siguientes: **concat()**, **copyWithin()**, **every()**, **filter()**, **flat()**, **flatMap()**, **forEach()**, **indexOf()**, **lastIndexOf()**, **map()**, **reduce()**, **reduceRight()**, **reverse()**, **slice()**, **some()**, **sort()**, y **splice()**. Los métodos de iteración como **forEach** no visitan las ranuras vacías en absoluto. Otros métodos, como **concat**, **copyWithin**, etc., conservan las ranuras vacías al hacer la copia, por lo que al final el array sigue siendo disperso.

JS

```
const colors = ["red", "yellow", "blue"];
colors[5] = "purple";
colors.forEach((item, index) => {
  console.log(`${index}: ${item}`);
});
// Output:
// 0: red
// 1: yellow
// 2: blue
// 5: purple

colors.reverse(); // ['purple', empty x 2, 'blue', 'yellow', 'red']
```

Los métodos más recientes (por ejemplo, **keys**) no tratan las ranuras vacías de forma especial y las tratan como si contuvieran **undefined**. Los métodos que combinan ranuras vacías con **undefined** elementos incluyen los siguientes:

`entries()`, `fill()`, `find()`, `findIndex()`, `findLast()`, `findLastIndex()`, `includes()`, `join()`, `keys()`, `toLocaleString()`, `toReversed()`, `toSorted()`, `toSpliced()`, `values()`, y `with()`.

JS

```
const colors = ["red", "yellow", "blue"];
colors[5] = "purple";
const iterator = colors.keys();
for (const key of iterator) {
  console.log(`${key}: ${colors[key]}`);
}
// Output
// 0: red
// 1: yellow
// 2: blue
// 3: undefined
// 4: undefined
// 5: purple

const newColors = colors.toReversed(); // ['purple', undefined, undefined, 'blue', 'yellow', 'red']
```

Métodos de copia y métodos de mutación

Algunos métodos no modifican el array existente sobre el que se invocaron, sino que devuelven un nuevo array. Para ello, primero construyen un nuevo array y luego lo rellenan con elementos. La copia siempre se realiza *de forma superficial*; el método nunca copia nada más allá del array creado inicialmente. Los elementos del array original se copian en el nuevo array de la siguiente manera:

- Objetos: la referencia al objeto se copia en el nuevo array. Tanto el array original como el nuevo hacen referencia al mismo objeto. Es decir, si se modifica un objeto referenciado, los cambios serán visibles tanto para el array nuevo como para el original.
- Tipos primitivos como cadenas, números y booleanos (no objetos `String`, `Number`, y `Boolean`): sus valores se copian en el nuevo array.

Otros métodos modifican el array sobre el que se llamó al método, en cuyo caso su valor de retorno difiere según el método: a veces una referencia al mismo array, a veces la longitud del nuevo array.

Los siguientes métodos crean nuevos arreglos accediendo `this.constructor[Symbol.species]` a para determinar el constructor a utilizar: `concat()`, `filter()`, `flat()`, `flatMap()`, `map()`, `slice()`, y `splice()` (para construir el arreglo de elementos eliminados que se devuelve).

Los siguientes métodos siempre crean nuevos arrays con el `Array` constructor base: `toReversed()`, `toSorted()`, `toSpliced()`, y `with()`.

La siguiente tabla enumera los métodos que modifican el array original y la alternativa correspondiente que no lo modifica:

Método de mutación	Alternativa no mutagénica
<code>copyWithin()</code>	No existe una alternativa de un solo método.
<code>fill()</code>	No existe una alternativa de un solo método.
<code>pop()</code>	<code>slice(0, -1)</code>
<code>push(v1, v2)</code>	<code>concat([v1, v2])</code>
<code>reverse()</code>	<code>toReversed()</code>
<code>shift()</code>	<code>slice(1)</code>
<code>sort()</code>	<code>toSorted()</code>
<code>splice()</code>	<code>toSpliced()</code>
<code>unshift(v1, v2)</code>	<code>toSpliced(0, 0, v1, v2)</code>

Una forma sencilla de convertir un método que modifica un método en una alternativa que no lo hace es usar la [sintaxis de propagación](#) o `slice()` crear primero una copia:

JS

```
arr.copyWithin(0, 1, 2); // mutates arr
const arr2 = arr.slice().copyWithin(0, 1, 2); // does not mutate arr
const arr3 = [...arr].copyWithin(0, 1, 2); // does not mutate arr
```

Métodos iterativos

Muchos métodos de array aceptan una función de devolución de llamada como argumento. Esta función se ejecuta secuencialmente y como máximo una vez por cada elemento del array, y su valor de retorno se utiliza para determinar el valor de retorno del método. Todos comparten la misma firma:

JS

```
method(callbackFn, thisArg)
```

Donde `callbackFn` se necesitan tres argumentos:

`element`

El elemento que se está procesando actualmente en el array.

`index`

El índice del elemento que se está procesando actualmente en el array.

`array`

El array sobre el que se llamó al método.

El `callbackFn` resultado esperado depende del método de matriz que se haya llamado.

The `thisArg` argument (defaults to `undefined`) will be used as the `this` value when calling `callbackFn`. The `this` value ultimately observable by `callbackFn` is determined according to [the usual rules](#): if `callbackFn` is `non-strict`, primitive `this` values are wrapped into objects, and `undefined` / `null` is substituted with `globalThis`. The `thisArg` argument is irrelevant for any `callbackFn` defined with an `arrow function`, as arrow functions don't have their own `this` [binding](#).

The `array` argument passed to `callbackFn` is most useful if you want to read another index during iteration, because you may not always have an existing variable that refers to the current array. You should generally not mutate the array during iteration (see [mutating initial array in iterative methods](#)), but you can also use this argument to do so. The `array` argument is *not* the array that is being built, in the case of methods like `map()`, `filter()`, and `flatMap()` — there is no way to access the array being built from the callback function.

All iterative methods are [copying](#) and [generic](#), although they behave differently with [empty slots](#).

The following methods are iterative: `every()`, `filter()`, `find()`, `findIndex()`, `findLast()`, `findLastIndex()`, `flatMap()`, `forEach()`, `map()`, and `some()`.

In particular, `every()`, `find()`, `findIndex()`, `findLast()`, `findLastIndex()`, and `some()` do not always invoke `callbackFn` on every element — they stop iteration as soon as the return value is determined.

The `reduce()` and `reduceRight()` methods also take a callback function and run it at most once for each element in the array, but they have slightly different signatures from typical iterative methods (for example, they don't accept `thisArg`).

The `sort()` method also takes a callback function, but it is not an iterative method. It mutates the array in-place, doesn't accept `thisArg`, and may invoke the callback multiple times on an index.

Iterative methods iterate the array like the following (with a lot of technical details omitted):

JS

```
function method(callbackFn, thisArg) {
  const length = this.length;
  for (let i = 0; i < length; i++) {
    if (i in this) {
      const result = callbackFn.call(thisArg, this[i], i, this);
      // Do something with result; maybe return early
    }
  }
}
```

Note the following:

1. Not all methods do the `i in this` test. The `find`, `findIndex`, `findLast`, and `findLastIndex` methods do not, but other methods do.
2. The `length` is memorized before the loop starts. This affects how insertions and deletions during iteration are handled (see [mutating initial array in iterative methods](#)).

3. The method doesn't memorize the array contents, so if any index is modified during iteration, the new value might be observed.
4. The code above iterates the array in ascending order of index. Some methods iterate in descending order of index (for (let i = length - 1; i >= 0; i--): `reduceRight()`, `findLast()`, and `findLastIndex()`).
5. `reduce` and `reduceRight` have slightly different signatures and do not always start at the first/last element.

Generic array methods

Array methods are always generic — they don't access any internal data of the array object. They only access the array elements through the `length` property and the indexed elements. This means that they can be called on array-like objects as well.

JS

```
const arrayLike = {
  0: "a",
  1: "b",
  length: 2,
};
console.log(Array.prototype.join.call(arrayLike, "+")); // 'a+b'
```

Normalization of the length property

The `length` property is converted to an integer and then clamped to the range between 0 and $2^{53} - 1$. `NaN` becomes 0, so even when `length` is not present or is `undefined`, it behaves as if it has value 0.

The language avoids setting `length` to an unsafe integer. All built-in methods will throw a `TypeError` if `length` will be set to a number greater than $2^{53} - 1$. However, because the `length` property of arrays throws an error if it's set to greater than $2^{32} - 1$, the safe integer threshold is usually not reached unless the method is called on a non-array object.

JS

```
Array.prototype.flat.call({}); // []
```

Some array methods set the `length` property of the array object. They always set the value after normalization, so `length` always ends as an integer.

JS

```
const a = { length: 0.7 };
Array.prototype.push.call(a);
console.log(a.length); // 0
```

Array-like objects

The term *array-like object* refers to any object that doesn't throw during the `length` conversion process described above. In practice, such object is expected to actually have a `length` property and to have indexed elements in the range `0` to `length - 1`. (If it doesn't have all indices, it will be functionally equivalent to a *sparse array*.) Any integer index less than zero or greater than `length - 1` is ignored when an array method operates on an array-like object.

Many DOM objects are array-like — for example, `NodeList` and `HTMLCollection`. The `arguments` object is also array-like. You can call array methods on them even if they don't have these methods themselves.

JS

```
function f() {  
  console.log(Array.prototype.join.call(arguments, "+"));  
}  
  
f("a", "b"); // 'a+b'
```

Constructor

`Array()`

Creates a new `Array` object.

Static properties

`Array[Symbol.species]`

Returns the `Array` constructor.

Static methods

`Array.from()`

Creates a new `Array` instance from an iterable or array-like object.

`Array.fromAsync()`

Creates a new `Array` instance from an async iterable, iterable, or array-like object.

`Array.isArray()`

Returns `true` if the argument is an array, or `false` otherwise.

`Array.of()`

Creates a new `Array` instance with a variable number of arguments, regardless of number or type of the arguments.

Instance properties

These properties are defined on `Array.prototype` and shared by all `Array` instances.

`Array.prototype.constructor`

The constructor function that created the instance object. For `Array` instances, the initial value is the `Array` constructor.

`Array.prototype[Symbol.unscopables]`

Contains property names that were not included in the ECMAScript standard prior to the ES2015 version and that are ignored for `with` statement-binding purposes.

These properties are own properties of each `Array` instance.

`length`

Reflects the number of elements in an array.

Instance methods

`Array.prototype.at()`

Returns the array item at the given index. Accepts negative integers, which count back from the last item.

`Array.prototype.concat()`

Returns a new array that is the calling array joined with other array(s) and/or value(s).

`Array.prototype.copyWithin()`

Copies a sequence of array elements within an array.

`Array.prototype.entries()`

Returns a new *array iterator* object that contains the key/value pairs for each index in an array.

`Array.prototype.every()`

Returns `false` if it finds an element in the array that does not satisfy the provided testing function. Otherwise, it returns `true`.

`Array.prototype.fill()`

Fills all the elements of an array from a start index to an end index with a static value.

`Array.prototype.filter()`

Returns a new array containing all elements of the calling array for which the provided filtering function returns `true`.

`Array.prototype.find()`

Returns the value of the first element in the array that satisfies the provided testing function, or `undefined` if no appropriate element is found.

`Array.prototype.findIndex()`

Returns the index of the first element in the array that satisfies the provided testing function, or `-1` if no appropriate element was found.

`Array.prototype.findLast()`

Returns the value of the last element in the array that satisfies the provided testing function, or `undefined` if no appropriate element is found.

`Array.prototype.findLastIndex()`

Returns the index of the last element in the array that satisfies the provided testing function, or `-1` if no appropriate element was found.

`Array.prototype.flat()`

Returns a new array with all sub-array elements concatenated into it recursively up to the specified depth.

`Array.prototype.flatMap()`

Returns a new array formed by applying a given callback function to each element of the calling array, and then flattening the result by one level.

`Array.prototype.forEach()`

Calls a function for each element in the calling array.

`Array.prototype.includes()`

Determines whether the calling array contains a value, returning `true` or `false` as appropriate.

`Array.prototype.indexOf()`

Returns the first (least) index at which a given element can be found in the calling array.

`Array.prototype.join()`

Joins all elements of an array into a string.

`Array.prototype.keys()`

Returns a new *array iterator* that contains the keys for each index in the calling array.

`Array.prototype.lastIndexOf()`

Returns the last (greatest) index at which a given element can be found in the calling array, or `-1` if none is found.

`Array.prototype.map()`

Returns a new array containing the results of invoking a function on every element in the calling array.

`Array.prototype.pop()`

Removes the last element from an array and returns that element.

`Array.prototype.push()`

Adds one or more elements to the end of an array, and returns the new `length` of the array.

`Array.prototype.reduce()`

Executes a user-supplied "reducer" callback function on each element of the array (from left to right), to reduce it to a single value.

`Array.prototype.reduceRight()`

Executes a user-supplied "reducer" callback function on each element of the array (from right to left), to reduce it to a single value.

`Array.prototype.reverse()`

Reverses the order of the elements of an array *in place*. (First becomes the last, last becomes first.)

`Array.prototype.shift()`

Removes the first element from an array and returns that element.

`Array.prototype.slice()`

Extracts a section of the calling array and returns a new array.

`Array.prototype.some()`

Returns `true` if it finds an element in the array that satisfies the provided testing function. Otherwise, it returns `false`.

`Array.prototype.sort()`

Sorts the elements of an array in place and returns the array.

`Array.prototype.splice()`

Adds and/or removes elements from an array.

`Array.prototype.toLocaleString()`

Returns a localized string representing the calling array and its elements. Overrides the `Object.prototype.toLocaleString()` method.

`Array.prototype.toReversed()`

Returns a new array with the elements in reversed order, without modifying the original array.

`Array.prototype.toSorted()`

Returns a new array with the elements sorted in ascending order, without modifying the original array.

`Array.prototype.toSpliced()`

Returns a new array with some elements removed and/or replaced at a given index, without modifying the original array.

`Array.prototype.toString()`

Returns a string representing the calling array and its elements. Overrides the `Object.prototype.toString()` method.

`Array.prototype.unshift()`

Adds one or more elements to the front of an array, and returns the new `length` of the array.

`Array.prototype.values()`

Returns a new *array iterator* object that contains the values for each index in the array.

`Array.prototype.with()`

Returns a new array with the element at the given index replaced with the given value, without modifying the original array.

`Array.prototype[Symbol.iterator]()`

An alias for the `values()` method by default.

Examples

This section provides some examples of common array operations in JavaScript.

 **Note:** If you're not yet familiar with array basics, consider first reading [JavaScript First Steps: Arrays](#), which explains what arrays are, and includes other examples of common array operations.

Create an array

This example shows three ways to create new array: first using [array literal notation](#), then using the `Array()` constructor, and finally using `String.prototype.split()` to build the array from a string.

JS

```
// 'fruits' array created using array literal notation.
const fruits = ["Apple", "Banana"];
console.log(fruits.length);
// 2

// 'fruits2' array created using the Array() constructor.
const fruits2 = new Array("Apple", "Banana");
console.log(fruits2.length);
// 2

// 'fruits3' array created using String.prototype.split().
const fruits3 = "Apple, Banana".split(", ");
console.log(fruits3.length);
// 2
```

Create a string from an array

This example uses the `join()` method to create a string from the `fruits` array.

JS

```
const fruits = ["Apple", "Banana"];
const fruitsString = fruits.join(", ");
console.log(fruitsString);
// "Apple, Banana"
```

Access an array item by its index

This example shows how to access items in the `fruits` array by specifying the index number of their position in the array.

JS

```
const fruits = ["Apple", "Banana"];

// The index of an array's first element is always 0.
fruits[0]; // Apple

// The index of an array's second element is always 1.
fruits[1]; // Banana

// The index of an array's last element is always one
// less than the length of the array.
fruits[fruits.length - 1]; // Banana

// Using an index number larger than the array's length
// returns 'undefined'.
fruits[99]; // undefined
```

Find the index of an item in an array

This example uses the `indexOf()` method to find the position (index) of the string `"Banana"` in the `fruits` array.

JS

```
const fruits = ["Apple", "Banana"];
console.log(fruits.indexOf("Banana"));
// 1
```

Check if an array contains a certain item

This example shows two ways to check if the `fruits` array contains `"Banana"` and `"Cherry"`: first with the `includes()` method, and then with the `indexOf()` method to test for an index value that's not `-1`.

JS

```
const fruits = ["Apple", "Banana"];

fruits.includes("Banana"); // true
fruits.includes("Cherry"); // false

// If indexOf() doesn't return -1, the array contains the given item.
fruits.indexOf("Banana") !== -1; // true
fruits.indexOf("Cherry") !== -1; // false
```

Append an item to an array

This example uses the `push()` method to append a new string to the `fruits` array.

JS

```
const fruits = ["Apple", "Banana"];
const newLength = fruits.push("Orange");
console.log(fruits);
// ["Apple", "Banana", "Orange"]
console.log(newLength);
// 3
```

Remove the last item from an array

This example uses the `pop()` method to remove the last item from the `fruits` array.

JS

```
const fruits = ["Apple", "Banana", "Orange"];
const removedItem = fruits.pop();
console.log(fruits);
// ["Apple", "Banana"]
console.log(removedItem);
// Orange
```

 **Note:** `pop()` can only be used to remove the last item from an array. To remove multiple items from the end of an array, see the next example.

Remove multiple items from the end of an array

This example uses the `splice()` method to remove the last 3 items from the `fruits` array.

JS

```
const fruits = ["Apple", "Banana", "Strawberry", "Mango", "Cherry"];
const start = -3;
const removedItems = fruits.splice(start);
console.log(fruits);
// ["Apple", "Banana"]
console.log(removedItems);
// ["Strawberry", "Mango", "Cherry"]
```

Truncate an array down to just its first N items

This example uses the `splice()` method to truncate the `fruits` array down to just its first 2 items.

JS

```
const fruits = ["Apple", "Banana", "Strawberry", "Mango", "Cherry"];
const start = 2;
const removedItems = fruits.splice(start);
console.log(fruits);
// ["Apple", "Banana"]
console.log(removedItems);
// ["Strawberry", "Mango", "Cherry"]
```

Remove the first item from an array

This example uses the `shift()` method to remove the first item from the `fruits` array.

JS

```
const fruits = ["Apple", "Banana"];
const removedItem = fruits.shift();
console.log(fruits);
// ["Banana"]
console.log(removedItem);
// Apple
```

 **Note:** `shift()` can only be used to remove the first item from an array. To remove multiple items from the beginning of an array, see the next example.

Remove multiple items from the beginning of an array

This example uses the `splice()` method to remove the first 3 items from the `fruits` array.

JS

```
const fruits = ["Apple", "Strawberry", "Cherry", "Banana", "Mango"];
const start = 0;
const deleteCount = 3;
const removedItems = fruits.splice(start, deleteCount);
console.log(fruits);
// ["Banana", "Mango"]
console.log(removedItems);
// ["Apple", "Strawberry", "Cherry"]
```

Add a new first item to an array

This example uses the `unshift()` method to add, at index `0`, a new item to the `fruits` array — making it the new first item in the array.

JS

```
const fruits = ["Banana", "Mango"];
const newLength = fruits.unshift("Strawberry");
console.log(fruits);
// ["Strawberry", "Banana", "Mango"]
console.log(newLength);
// 3
```

Remove a single item by index

This example uses the `splice()` method to remove the string `"Banana"` from the `fruits` array — by specifying the index position of `"Banana"`.

JS

```
const fruits = ["Strawberry", "Banana", "Mango"];
const start = fruits.indexOf("Banana");
const deleteCount = 1;
const removedItems = fruits.splice(start, deleteCount);
console.log(fruits);
// ["Strawberry", "Mango"]
console.log(removedItems);
// ["Banana"]
```

Remove multiple items by index

This example uses the `splice()` method to remove the strings `"Banana"` and `"Strawberry"` from the `fruits` array — by specifying the index position of `"Banana"`, along with a count of the number of total items to remove.

JS

```
const fruits = ["Apple", "Banana", "Strawberry", "Mango"];
const start = 1;
const deleteCount = 2;
const removedItems = fruits.splice(start, deleteCount);
console.log(fruits);
// ["Apple", "Mango"]
console.log(removedItems);
// ["Banana", "Strawberry"]
```

Replace multiple items in an array

This example uses the `splice()` method to replace the last 2 items in the `fruits` array with new items.

JS

```
const fruits = ["Apple", "Banana", "Strawberry"];
const start = -2;
const deleteCount = 2;
const removedItems = fruits.splice(start, deleteCount, "Mango", "Cherry");
console.log(fruits);
// ["Apple", "Mango", "Cherry"]
console.log(removedItems);
// ["Banana", "Strawberry"]
```

Iterate over an array

This example uses a `for...of` loop to iterate over the `fruits` array, logging each item to the console.

JS

```
const fruits = ["Apple", "Mango", "Cherry"];
for (const fruit of fruits) {
  console.log(fruit);
}
// Apple
// Mango
// Cherry
```

But `for...of` is just one of many ways to iterate over any array; for more ways, see [Loops and iteration](#), and see the documentation for the `every()`, `filter()`, `flatMap()`, `map()`, `reduce()`, and `reduceRight()` methods — and see the next example, which uses the `forEach()` method.

Call a function on each element in an array

This example uses the `forEach()` method to call a function on each element in the `fruits` array; the function causes each item to be logged to the console, along with the item's index number.

JS

```
const fruits = ["Apple", "Mango", "Cherry"];
fruits.forEach((item, index, array) => {
  console.log(item, index);
});
// Apple 0
// Mango 1
// Cherry 2
```

Merge multiple arrays together

This example uses the `concat()` method to merge the `fruits` array with a `moreFruits` array, to produce a new `combinedFruits` array. Notice that `fruits` and `moreFruits` remain unchanged.

JS

```
const fruits = ["Apple", "Banana", "Strawberry"];
const moreFruits = ["Mango", "Cherry"];
const combinedFruits = fruits.concat(moreFruits);
console.log(combinedFruits);
// ["Apple", "Banana", "Strawberry", "Mango", "Cherry"]

// The 'fruits' array remains unchanged.
console.log(fruits);
// ["Apple", "Banana", "Strawberry"]

// The 'moreFruits' array also remains unchanged.
console.log(moreFruits);
// ["Mango", "Cherry"]
```

Copy an array

This example shows three ways to create a new array from the existing `fruits` array: first by using [spread syntax](#), then by using the `from()` method, and then by using the `slice()` method.

JS

```
const fruits = ["Strawberry", "Mango"];

// Create a copy using spread syntax.
const fruitsCopy = [...fruits];
// ["Strawberry", "Mango"]

// Create a copy using the from() method.
const fruitsCopy2 = Array.from(fruits);
// ["Strawberry", "Mango"]

// Create a copy using the slice() method.
const fruitsCopy3 = fruits.slice();
// ["Strawberry", "Mango"]
```

All built-in array-copy operations ([spread syntax](#), `Array.from()`, `Array.prototype.slice()`, and `Array.prototype.concat()`) create [shallow copies](#). If you instead want a [deep copy](#) of an array, you can use `JSON.stringify()` to convert the array to a JSON string, and then `JSON.parse()` to convert the string back into a new array that's completely independent from the original array.

JS

```
const fruitsDeepCopy = JSON.parse(JSON.stringify(fruits));
```

You can also create deep copies using the `structuredClone()` method, which has the advantage of allowing [transferable objects](#) in the source to be *transferred* to the new copy, rather than just cloned.

Finally, it's important to understand that assigning an existing array to a new variable doesn't create a copy of either the array or its elements. Instead the new variable is just a reference, or alias, to the original array; that is, the original array's name and the new variable name are just two names for the exact same object (and so will always evaluate as [strictly equivalent](#)). Therefore, if you make any changes at all either to the value of the original array or to the value of the new variable, the other will change, too:

JS

```
const fruits = ["Strawberry", "Mango"];
const fruitsAlias = fruits;
// 'fruits' and 'fruitsAlias' are the same object, strictly equivalent.
fruits === fruitsAlias; // true
// Any changes to the 'fruits' array change 'fruitsAlias' too.
fruits.unshift("Apple", "Banana");
console.log(fruits);
// ['Apple', 'Banana', 'Strawberry', 'Mango']
console.log(fruitsAlias);
// ['Apple', 'Banana', 'Strawberry', 'Mango']
```

Creating a two-dimensional array

The following creates a chessboard as a two-dimensional array of strings. The first move is made by copying the `'p'` in `board[6][4]` to `board[4][4]`. The old position at `[6][4]` is made blank.

JS

```
const board = [
  ["R", "N", "B", "Q", "K", "B", "N", "R"],
  ["P", "P", "P", "P", "P", "P", "P", "P"],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  ["p", "p", "p", "p", "p", "p", "p", "p"],
  ["r", "n", "b", "q", "k", "b", "n", "r"],
];

console.log(`${board.join("\n")}\n\n`);
```

```
// Move King's Pawn forward 2
board[4][4] = board[6][4];
board[6][4] = " ";
console.log(board.join("\n"));
```

Here is the output:

```
R,N,B,Q,K,B,N,R
P,P,P,P,P,P,P,P
, , , , , , ,
, , , , , , ,
, , , , , , ,
, , , , , , ,
p,p,p,p,p,p,p,p
r,n,b,q,k,b,n,r
```

```
R,N,B,Q,K,B,N,R
P,P,P,P,P,P,P,P
, , , , , , ,
, , , , , , ,
, , , ,P, , ,
, , , , , , ,
p,p,p,p, ,p,p,p
r,n,b,q,k,b,n,r
```

Using an array to tabulate a set of values

JS

```
const values = [];
for (let x = 0; x < 10; x++) {
  values.push([2 ** x, 2 * x ** 2]);
}
console.table(values);
```

Results in

```
// The first column is the index
0 1 0
1 2 2
2 4 8
3 8 18
4 16 32
5 32 50
6 64 72
7 128 98
```

```
8 256 128
9 512 162
```

Creating an array using the result of a match

The result of a match between a `RegExp` and a string can create a JavaScript array that has properties and elements which provide information about the match. Such an array is returned by `RegExp.prototype.exec()` and `String.prototype.match()`.

For example:

JS

```
// Match one d followed by one or more b's followed by one d
// Remember matched b's and the following d
// Ignore case

const myRe = /d(b+)(d)/i;
const execResult = myRe.exec("cdbBdbsbz");

console.log(execResult.input); // 'cdbBdbsbz'
console.log(execResult.index); // 1
console.log(execResult); // [ "dbBd", "bB", "d" ]
```

For more information about the result of a match, see the `RegExp.prototype.exec()` and `String.prototype.match()` pages.

Mutating initial array in iterative methods

Iterative methods do not mutate the array on which it is called, but the function provided as `callbackFn` can. The key principle to remember is that only indexes between `0` and `arrayLength - 1` are visited, where `arrayLength` is the length of the array at the time the array method was first called, but the element passed to the callback is the value at the time the index is visited. Therefore:

- `callbackFn` will not visit any elements added beyond the array's initial length when the call to the iterative method began.
- Changes to already-visited indexes do not cause `callbackFn` to be invoked on them again.
- If an existing, yet-unvisited element of the array is changed by `callbackFn`, its value passed to the `callbackFn` will be the value at the time that element gets visited. Removed elements are not visited.

 **Warning:** Concurrent modifications of the kind described above frequently lead to hard-to-understand code and are generally to be avoided (except in special cases).

The following examples use the `forEach` method as an example, but other methods that visit indexes in ascending order work in the same way. We will first define a helper function:

JS

```
function testSideEffect(effect) {  
  const arr = ["e1", "e2", "e3", "e4"];  
  arr.forEach((elem, index, arr) => {  
    console.log(`array: [${arr.join(", ")}], index: ${index}, elem: ${elem}`);  
    effect(arr, index);  
  });  
  console.log(`Final array: [${arr.join(", ")}]`);  
}
```

Modification to indexes not visited yet will be visible once the index is reached:

JS

```
testSideEffect((arr, index) => {  
  if (index + 1 < arr.length) arr[index + 1] += "*";  
});  
// array: [e1, e2, e3, e4], index: 0, elem: e1  
// array: [e1, e2*, e3, e4], index: 1, elem: e2*  
// array: [e1, e2*, e3*, e4], index: 2, elem: e3*  
// array: [e1, e2*, e3*, e4*], index: 3, elem: e4*  
// Final array: [e1, e2*, e3*, e4*]
```

Modification to already visited indexes does not change iteration behavior, although the array will be different afterwards:

JS

```
testSideEffect((arr, index) => {  
  if (index > 0) arr[index - 1] += "*";  
});  
// array: [e1, e2, e3, e4], index: 0, elem: e1  
// array: [e1, e2, e3, e4], index: 1, elem: e2  
// array: [e1*, e2, e3, e4], index: 2, elem: e3  
// array: [e1*, e2*, e3, e4], index: 3, elem: e4  
// Final array: [e1*, e2*, e3*, e4]
```

Inserting n elements at unvisited indexes that are less than the initial array length will make them be visited. The last n elements in the original array that now have index greater than the initial array length will not be visited:

JS

```
testSideEffect((arr, index) => {  
  if (index === 1) arr.splice(2, 0, "new");  
});
```

```
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4], index: 1, elem: e2
// array: [e1, e2, new, e3, e4], index: 2, elem: new
// array: [e1, e2, new, e3, e4], index: 3, elem: e3
// Final array: [e1, e2, new, e3, e4]
// e4 is not visited because it now has index 4
```

Inserting n elements with index greater than the initial array length will not make them be visited:

JS

```
testSideEffect((arr) => arr.push("new"));
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4, new], index: 1, elem: e2
// array: [e1, e2, e3, e4, new, new], index: 2, elem: e3
// array: [e1, e2, e3, e4, new, new, new], index: 3, elem: e4
// Final array: [e1, e2, e3, e4, new, new, new, new]
```

Inserting n elements at already visited indexes will not make them be visited, but it shifts remaining elements back by n , so the current index and the $n - 1$ elements before it are visited again:

JS

```
testSideEffect((arr, index) => arr.splice(index, 0, "new"));
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [new, e1, e2, e3, e4], index: 1, elem: e1
// array: [new, new, e1, e2, e3, e4], index: 2, elem: e1
// array: [new, new, new, e1, e2, e3, e4], index: 3, elem: e1
// Final array: [new, new, new, new, e1, e2, e3, e4]
// e1 keeps getting visited because it keeps getting shifted back
```

Deleting n elements at unvisited indexes will make them not be visited anymore. Because the array has shrunk, the last n iterations will visit out-of-bounds indexes. If the method ignores non-existent indexes (see [array methods and empty slots](#)), the last n iterations will be skipped; otherwise, they will receive `undefined`:

JS

```
testSideEffect((arr, index) => {
  if (index === 1) arr.splice(2, 1);
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4], index: 1, elem: e2
// array: [e1, e2, e4], index: 2, elem: e4
// Final array: [e1, e2, e4]
// Does not visit index 3 because it's out-of-bounds
```

```
// Compare this with find(), which treats nonexistent indexes as undefined:
const arr2 = ["e1", "e2", "e3", "e4"];
arr2.find((elem, index, arr) => {
  console.log(`array: [${arr.join(", ")}], index: ${index}, elem: ${elem}`);
  if (index === 1) arr.splice(2, 1);
  return false;
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4], index: 1, elem: e2
// array: [e1, e2, e4], index: 2, elem: e4
// array: [e1, e2, e4], index: 3, elem: undefined
```

Deleting n elements at already visited indexes does not change the fact that they were visited before they get deleted. Because the array has shrunk, the next n elements after the current index are skipped. If the method ignores non-existent indexes, the last n iterations will be skipped; otherwise, they will receive `undefined`:

JS

```
testSideEffect((arr, index) => arr.splice(index, 1));
// array: [e1, e2, e3, e4], index: 0, elem: e1
// Does not visit e2 because e2 now has index 0, which has already been visited
// array: [e2, e3, e4], index: 1, elem: e3
// Does not visit e4 because e4 now has index 1, which has already been visited
// Final array: [e2, e4]
// Index 2 is out-of-bounds, so it's not visited

// Compare this with find(), which treats nonexistent indexes as undefined:
const arr2 = ["e1", "e2", "e3", "e4"];
arr2.find((elem, index, arr) => {
  console.log(`array: [${arr.join(", ")}], index: ${index}, elem: ${elem}`);
  arr.splice(index, 1);
  return false;
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e2, e3, e4], index: 1, elem: e3
// array: [e2, e4], index: 2, elem: undefined
// array: [e2, e4], index: 3, elem: undefined
```

For methods that iterate in descending order of index, insertion causes elements to be skipped, and deletion causes elements to be visited multiple times. Adjust the code above yourself to see the effects.

Specifications

Specification

ECMAScript® 2027 Language Specification
sec-array-objects [↗](#)

Browser compatibility

[Informar de problemas con estos datos de compatibilidad ↗](#) • [Ver datos en GitHub ↗](#)

															
	Cromo	Borde	Firefox	Ópera	Safari	Chrome para Android	Firefox para Android	Opera Android	Safari en iOS	Internet de Samsung	WebView Android	WebView en iOS	Bollo	Deno	Node.js
Array	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,3
[Símbolo.iterador]	✓ 38	✓ 12	✓ 36 ...	✓ 25	✓ 10	✓ 38	✓ 36 ...	✓ 25	✓ 10	✓ 3	✓ 38	✓ 10	✓ 1	✓ 1	✓ 0,3
[Símbolo.especie]	✓ 51	✓ 79	✓ 48	✓ 38	✓ 10	✓ 51	✓ 48	✓ 41	✓ 10	✓ 5	✓ 51	✓ 10	✓ 1	✓ 1	✓ 6,3
[Símbolos inescrutables]	✓ 38	✓ 12	✓ 48	✓ 25	✓ 10	✓ 38	✓ 48	✓ 25	✓ 10	✓ 3	✓ 38	✓ 10	✓ 1	✓ 1	✓ 0,3
Array(). constructor	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,3
at	✓ 92	✓ 92	✓ 90	✓ 78	✓ 15.4	✓ 92	✓ 90	✓ 65	✓ 15.4	✓ 16	✓ 92	✓ 15.4	✓ 1	✓ 1.12	✓ 16
concat	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,3
copyWithin	✓ 45	✓ 12	✓ 32	✓ 32	✓ 9	✓ 45	✓ 32	✓ 32	✓ 9	✓ 5	✓ 45	✓ 9	✓ 1	✓ 1	✓ 4
entries	✓ 38	✓ 12	✓ 28	✓ 25	✓ 8	✓ 38	✓ 28	✓ 25	✓ 8	✓ 3	✓ 38	✓ 8	✓ 1	✓ 1	✓ 0,3
every	✓ 1	✓ 12	✓ 1.5	✓ 9.5	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 4.4	✓ 1	✓ 1	✓ 1	✓ 0,3
fill	✓ 45	✓ 12	✓ 31	✓ 32	✓ 8	✓ 45	✓ 31	✓ 32	✓ 8	✓ 5	✓ 45	✓ 8	✓ 1	✓ 1	✓ 4

filter	✓ 1	✓ 12	✓ 1.5	✓ 9.5	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 4.4	✓ 1	✓ 1	✓ 1	✓ 0,
find	✓ 45	✓ 12	✓ 25	✓ 32	✓ 8	✓ 45	✓ 4	✓ 32	✓ 8	✓ 5	✓ 45	✓ 8	✓ 1	✓ 1	✓ 4
findIndex	✓ 45	✓ 12	✓ 25	✓ 32	✓ 8	✓ 45	✓ 4	✓ 32	✓ 8	✓ 5	✓ 45	✓ 8	✓ 1	✓ 1	✓ 4
findLast	✓ 97	✓ 97	✓ 104	✓ 83	✓ 15.4	✓ 97	✓ 104	✓ 68	✓ 15.4	✓ 18	✓ 97	✓ 15.4	✓ 1	✓ 1.16	✓ 1
findLastIndex	✓ 97	✓ 97	✓ 104	✓ 83	✓ 15.4	✓ 97	✓ 104	✓ 68	✓ 15.4	✓ 18	✓ 97	✓ 15.4	✓ 1	✓ 1.16	✓ 1
flat	✓ 69	✓ 79	✓ 62	✓ 56	✓ 12	✓ 69	✓ 62	✓ 48	✓ 12	✓ 10	✓ 69	✓ 12	✓ 1	✓ 1	✓ 1
flatMap	✓ 69	✓ 79	✓ 62	✓ 56	✓ 12	✓ 69	✓ 62	✓ 48	✓ 12	✓ 10	✓ 69	✓ 12	✓ 1	✓ 1	✓ 1
forEach	✓ 1	✓ 12	✓ 1.5	✓ 9.5	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 4.4	✓ 1	✓ 1	✓ 1	✓ 0,
from	✓ 45	✓ 12	✓ 32	✓ 32	✓ 9	✓ 45	✓ 32	✓ 32	✓ 9	✓ 5	✓ 45	✓ 9	✓ 1	✓ 1	✓ 4
fromAsync	✓ 121	✓ 121	✓ 115	✓ 107	✓ 16.4	✓ 121	✓ 115	✓ 81	✓ 16.4	✓ 25	✓ 121	✓ 16.4	✓ 1	✓ 1.38	✓ 2
includes	✓ 47	✓ 14	✓ 43	✓ 34	✓ 9	✓ 47	✓ 43	✓ 34	✓ 9	✓ 5	✓ 47	✓ 9	✓ 1	✓ 1	✓ 6
indexOf	✓ 1	✓ 12	✓ 1.5	✓ 9.5	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 4.4	✓ 1	✓ 1	✓ 1	✓ 0,
isArray	✓ 4	✓ 12	✓ 4	✓ 10.5	✓ 5	✓ 18	✓ 4	✓ 14	✓ 5	✓ 1	✓ 4.4	✓ 5	✓ 1	✓ 1	✓ 0,
join	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,
keys	✓ 38	✓ 12	✓ 28	✓ 25	✓ 8	✓ 38	✓ 28	✓ 25	✓ 8	✓ 3	✓ 38	✓ 8	✓ 1	✓ 1	✓ 0,
lastIndexOf	✓ 1	✓ 12	✓ 1.5	✓ 9.5	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 4.4	✓ 1	✓ 1	✓ 1	✓ 0,
length	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,
map	✓ 1	✓ 12	✓ 1.5	✓ 9.5	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 4.4	✓ 1	✓ 1	✓ 1	✓ 0,

of	✓ 45	✓ 12	✓ 25	✓ 26	✓ 9	✓ 39	✓ 25	✓ 26	✓ 9	✓ 4	✓ 39	✓ 9	✓ 1	✓ 1	✓ 4
pop	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,
push	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,
reduce	✓ 3	✓ 12	✓ 3	✓ 10.5	✓ 4	✓ 18	✓ 4	✓ 14	✓ 3.2	✓ 1	✓ 4.4	✓ 3.2	✓ 1	✓ 1	✓ 0,
reduceRight	✓ 3	✓ 12	✓ 3	✓ 10.5	✓ 4	✓ 18	✓ 4	✓ 14	✓ 3.2	✓ 1	✓ 4.4	✓ 3.2	✓ 1	✓ 1	✓ 0,
reverse	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,
shift	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,
slice	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,
some	✓ 1	✓ 12	✓ 1.5	✓ 9.5	✓ 3	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 4.4	✓ 1	✓ 1	✓ 1	✓ 0,
sort	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 4.4	✓ 1	✓ 1	✓ 1	✓ 0,
Clasificación estable	✓ 70	✓ 79	✓ 3	✓ 57	✓ 10.1	✓ 70	✓ 4	✓ 49	✓ 10.3	✓ 10	✓ 70	✓ 10.3	✓ 1	✓ 1	✓ 1
splice	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 4.4	✓ 1	✓ 1	✓ 1	✓ 0,
toLocaleString	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,
locales parámetro	✓ 24	✓ 79	✓ 52	✓ 15	✓ 7	✓ 25	✓ 56	✓ 14	✓ 7	✓ 2	✓ 4.4	✓ 7	✓ 1	✓ 1.8	✓ 1
options parámetro	✓ 24	✓ 79	✓ 52	✓ 15	✓ 7	✓ 25	✓ 56	✓ 14	✓ 7	✓ 2	✓ 4.4	✓ 7	✓ 1	✓ 1	✓ 0,
toReversed	✓ 110	✓ 110	✓ 115	✓ 96	✓ 16	✓ 110	✓ 115	✓ 74	✓ 16	✓ 21	✓ 110	✓ 16	✓ 1	✓ 1.31	✓ 2
toSorted	✓ 110	✓ 110	✓ 115	✓ 96	✓ 16	✓ 110	✓ 115	✓ 74	✓ 16	✓ 21	✓ 110	✓ 16	✓ 1	✓ 1.31	✓ 2
toSpliced	✓ 110	✓ 110	✓ 115	✓ 96	✓ 16	✓ 110	✓ 115	✓ 74	✓ 16	✓ 21	✓ 110	✓ 16	✓ 1	✓ 1.31	✓ 2

toString	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,
unshift	✓ 1	✓ 12	✓ 1	✓ 4	✓ 1	✓ 18	✓ 4	✓ 10.1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 1	✓ 0,
values	✓ 66	✓ 14	✓ 60	✓ 53	✓ 9	✓ 66	✓ 60	✓ 47	✓ 9	✓ 9	✓ 66	✓ 9	✓ 1	✓ 1	✓ 10 ..
with	✓ 110	✓ 110	✓ 115	✓ 96	✓ 16	✓ 110	✓ 115	✓ 74	✓ 16	✓ 21	✓ 110	✓ 16	✓ 1	✓ 1.31	✓ 2

✓ Soporte completo ⌚ Apoyo parcial 🐛 Utiliza un nombre no estándar.

... Contiene más información de compatibilidad.

See also

- [Indexed collections](#) guide
- [TypedArray](#)
- [ArrayBuffer](#)



Tu plan para una mejor internet.